

DL Assignment-5

Q1) How does deep learning contribute to the field of speech recognition, and what key advancements have played a pivotal role in enhancing the accuracy and efficiency of speech recognition systems?

A)

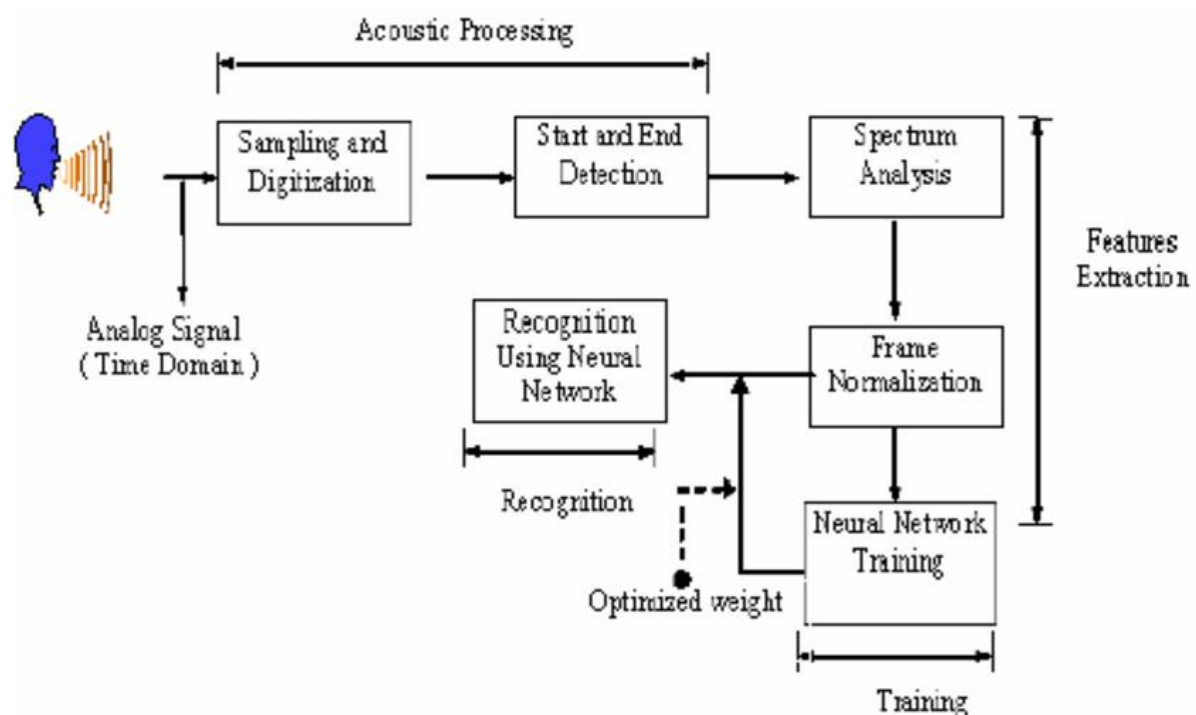
Deep Learning in Speech Recognition

Speech recognition (also called **Automatic Speech Recognition – ASR**) is the process of converting spoken language into text. Traditionally, ASR was based on **statistical models** such as:

- **Hidden Markov Models (HMMs)** – for modeling time-dependent sequences.
- **Gaussian Mixture Models (GMMs)** – for modeling acoustic features.

These systems worked decently but had **limitations**: they required handcrafted features (like MFCCs), struggled with variability in accents/noise, and didn't capture long-range dependencies well.

Deep learning changed everything by allowing **end-to-end learning** directly from raw audio or spectrograms, automatically learning features and mappings to text with much higher accuracy.



How Deep Learning Contributes

1. Automatic Feature Learning

- Traditional ASR required manual extraction of features (MFCCs, PLPs).
- **Convolutional Neural Networks (CNNs)** and **Recurrent Neural Networks (RNNs)** learn feature representations directly from spectrograms or raw waveforms.
- This reduces human effort and improves robustness to noise, accents, and variations.

2. Sequence Modeling

Speech is inherently sequential. Deep learning provides tools to capture temporal dependencies:

- **RNNs** (especially **LSTM** and **GRU**) can model long-term context in speech.
- **Bidirectional RNNs** capture both past and future context for better recognition.
- **Attention mechanisms** and **Transformers** allow the model to focus on relevant parts of the input sequence without being limited by sequential processing.

3. End-to-End Architectures

Instead of separate modules (acoustic model + language model + decoder), deep learning enabled **end-to-end models** that directly map audio to text. Examples:

- **CTC (Connectionist Temporal Classification)**: handles variable-length input/output alignment.
- **Seq2Seq with Attention**: models speech as a sequence-to-sequence translation problem.
- **Transformer-based models**: e.g., Google's **Speech-Transformer**, Facebook's **wav2vec 2.0**, OpenAI's **Whisper**.

4. Scalability with Data and Compute

Deep learning thrives with **big datasets** and **GPU/TPU acceleration**. Speech recognition improved drastically when large-scale datasets (e.g., LibriSpeech, CommonVoice, YouTube transcriptions) were combined with high-capacity deep networks.

Key Advancements in Deep Learning for Speech Recognition

1. Deep Neural Networks (DNNs) replacing GMMs

- Early 2010s: DNNs started being used as acoustic models instead of GMMs.
- Reduced Word Error Rate (WER) significantly.

2. LSTMs and GRUs (Handling long sequences)

- RNN-based models improved handling of temporal dependencies.
- **Bidirectional LSTMs** allowed the system to use both past and future speech context.

3. Connectionist Temporal Classification (CTC)

- Introduced by Alex Graves (2006, popularized in 2012).
- Allowed end-to-end training without pre-aligned input-output sequences.
- Example: Baidu's **Deep Speech** used CTC + RNNs to achieve state-of-the-art results.

4. Attention Mechanisms and Seq2Seq Models

- Attention solved the **alignment problem** between speech frames and text tokens.
- Enabled models to "focus" on important parts of the input.
- Widely used in **neural machine translation** and quickly adopted for ASR.

5. Transformers and Self-Attention

- Transformer models (Vaswani et al., 2017) revolutionized NLP and later speech recognition.
- **Speech-Transformer, wav2vec 2.0, Whisper** use self-attention to model long-range dependencies efficiently.
- Provide higher accuracy and robustness compared to RNNs.

6. Self-Supervised Learning

- Training on massive amounts of **unlabeled audio** has become crucial.
- Models like **wav2vec 2.0 (Facebook AI)** and **HuBERT** learn general-purpose speech representations from raw audio.
- Fine-tuning with smaller labeled datasets yields state-of-the-art results.

7. Multilingual and Zero-Shot Models

- Models like **Whisper (OpenAI)** and **XLS-R (Meta)** can handle **multiple languages** and even perform **speech-to-text translation**.
- This makes speech recognition accessible for low-resource languages.

Advantages of Deep Learning in ASR

- **Higher Accuracy:** Dramatic reduction in Word Error Rate (WER).
- **Noise Robustness:** Works better in real-world noisy environments.
- **End-to-End Learning:** No need for handcrafted features and separate modules.
- **Multilingual Capability:** A single model can recognize speech in many languages.

- **Adaptability:** Can fine-tune easily for domain-specific vocabulary (e.g., medical, legal).

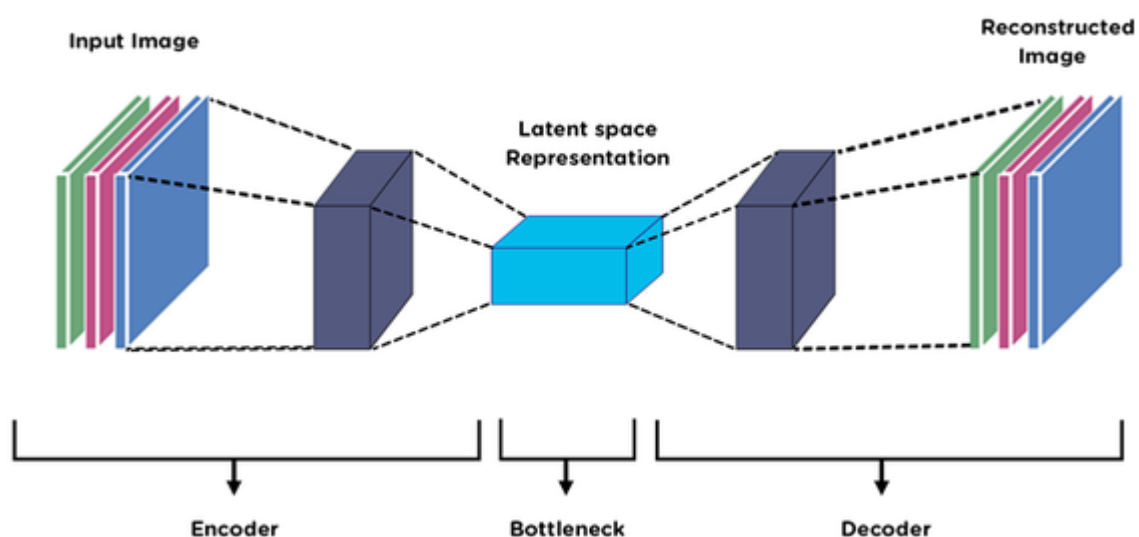
Example Pipeline with Deep Learning

1. Input audio → transformed into **spectrogram** or raw waveform.
2. CNN layers → extract local acoustic features.
3. RNN/Transformer layers → capture temporal dependencies.
4. CTC / Seq2Seq decoder → map features to character or word sequence.
5. Language model integration (optional) → improves fluency and correctness.
6. Output: Transcribed text.

Q2) Explain about Autoencoder types in detail and its applications for each type.

A)

An **autoencoder (AE)** is a neural network trained to reconstruct its input. It compresses input x to a lower-dimensional **latent code** $z = \text{Encoder}(x)$ and then reconstructs $\hat{x} = \text{Decoder}(z)$. The objective is typically to minimize a reconstruction loss (mean squared error for continuous data, cross-entropy for binary/normalized data). Because the network must compress and then reconstruct, the latent code often captures useful structure (features) about the data.



Core AE building blocks (quick recap)

- **Encoder** $f_\phi(x) \rightarrow$ produces latent representation z (vector or distribution parameters).
- **Latent space** $z \rightarrow$ bottleneck; size/structure determines capacity.
- **Decoder** $g_\theta(z) \rightarrow$ reconstructs $\hat{x}$.
- **Reconstruction loss** $L_{\text{rec}}(x, \hat{x})$ (MSE, BCE, etc.).
- Optional **regularizers** on z (sparsity, KL, adversarial loss, Jacobian penalty).

1) Vanilla / Undercomplete Autoencoder

What: A deterministic AE with a smaller latent dimension than input (bottleneck).

Why: Forces the model to learn compressed representations instead of trivial identity mapping.

Loss: Reconstruction loss (e.g., MSE).

Use-cases / Applications:

- Dimensionality reduction (alternative to PCA) — latent vectors as features.
- Pretraining for classification (learn representations, then fine-tune).
- Data compression (simple lossy compression).

Pros/Cons: Simple and effective; but can overfit if capacity is large, and features may be task-agnostic.

2) Overcomplete Autoencoder

What: Latent dimension $\geq$ input dimension. To prevent trivial copying, regularization (sparsity, denoising, contractive, dropout) is applied.

Why: Overcomplete + regularizer can learn richer, sparse features while preserving reconstruction.

Applications: Feature learning (sparse codes), modeling high-dimensional data when you want rich latent structure.

Pros/Cons: More flexible; needs careful regularization to avoid identity mapping.

3) Sparse Autoencoder

What: Encourage most latent units to be near zero (L1 penalty or KL-divergence to target activation).

Why: Enforces that only a small subset of neurons activate for any input $\rightarrow$ learned basis-like features (similar to sparse coding).

Loss: $L = L_{\text{rec}} + \lambda * \text{sparsity_penalty}$ (L1 or KL between mean activation and target).

Applications:

- Unsupervised feature extraction for images and signals.
- Anomaly detection: anomalies often fail to be encoded sparsely like normal data.
Pros/Cons: Good for interpretability and local feature detectors; tuning sparsity hyperparameter is important.

4) Denoising Autoencoder (DAE)

What: Train AE to reconstruct the **clean** input from a **corrupted** version (e.g., additive noise, masking).

Why: Forces the encoder to extract robust features that capture the underlying data manifold.

Loss: Reconstruction between clean x and decoder output from corrupted input $\tilde{x}$.

Applications:

- Image/audio denoising and inpainting.
- Robust feature learning for downstream classifiers.
- Pretraining (stacked DAEs historically used to initialize deep nets).
Pros/Cons: Very effective at learning robust representations; choice of corruption type matters.

5) Contractive Autoencoder (CAE)

What: Add a penalty on the Jacobian of the encoder ($\|\partial z / \partial x\|^2$) to make z insensitive to small input perturbations.

Why: Encourages representations that vary little for small changes in input $\rightarrow$ learns local manifold structure.

Applications: Manifold learning, robust feature learning, semi-supervised learning.

Pros/Cons: The Jacobian penalty can be computationally expensive for large models/dimensions.

6) Variational Autoencoder (VAE)

What: Probabilistic generative model. Encoder outputs parameters of a posterior distribution $q_\phi(z|x)$ (often Gaussian: mean & variance). Decoder models $p_\theta(x|z)$. Training maximizes the Evidence Lower Bound (ELBO).

Key math (ELBO):

$$\log p(x) \geq \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \parallel p(z))$$

Why: Learn a smooth latent space where sampling $z \sim p(z)$ and decoding produces realistic samples.

Applications:

- **Generative modeling** (image, speech, molecule generation).
- **Data augmentation** (sample synthetic but plausible examples).
- **Anomaly detection** (low reconstruction likelihood for anomalies).
- **Semi-supervised learning** (combine labeled and unlabeled via generative model).
Variants: β -VAE (disentanglement), Conditional VAE (cVAE—condition generation on labels), VQ-VAE (discrete latents).

Pros/Cons: Principled probabilistic model with continuous latent space and easy sampling; samples often blurrier than GANs; trade-off between reconstruction and latent regularization.

7) Adversarial Autoencoder (AAE)

What: Regularize the latent distribution by training a discriminator (GAN-style) to distinguish encoded latents from samples of a prior; encoder tries to fool the discriminator.

Why: Enforce any desired prior on z (e.g., Gaussian, mixture), combine AE reconstruction with adversarial regularization.

Applications: Generative modeling, semi-supervised learning (AAE variants can incorporate labels), imposing structured latent spaces (mixture models for clustering).

Pros/Cons: Combines pros of GANs and AEs; adversarial training instability is a concern.

8) Convolutional Autoencoder (CAE / Conv-AE)

What: Encoder/decoder built from convolutional and (transposed) convolutional layers instead of dense layers.

Why: Leverage spatial locality and parameter sharing for images/video; much fewer parameters and better inductive bias than dense AEs for grid data.

Applications:

- Image compression and reconstruction.
- Image denoising, inpainting, super-resolution (with architectural tweaks).
- Feature extraction for images (pretraining).
- Anomaly detection in images (industrial defect detection).

Pros/Cons: Excellent for images; must design decoder carefully to avoid checkerboard artifacts (use upsampling + conv).

9) Recurrent / Sequence Autoencoder (Seq-AE)

What: Encoder and decoder use RNN/LSTM/GRU. Encoder processes a variable-length input sequence into a vector z ; decoder reconstructs a sequence from z . Can be **many-to-one-to-many** or sequence-to-sequence.

Why: For sequential data (text, audio, time-series) you need models that capture temporal dependencies.

Applications:

- Sequence embedding (sentence/document embeddings).
- Anomaly detection in time series (e.g., sensor monitoring).
- Representation learning for speech/audio (pretraining).
- Sequence prediction and compression.

Pros/Cons: Effective for medium-length sequences; limited for very long contexts compared with Transformers.

10) Stacked / Deep Autoencoder

What: Multiple encoder/decoder layers (deep networks). Often pre-trained layer-wise (historical) or trained end-to-end.

Why: Learn hierarchical features of increasing abstraction.

Applications: Feature pretraining, dimensionality reduction for high-dimensional datasets, initializing deep classifiers.

Pros/Cons: More expressive but may require more data/regularization.

11) Vector Quantized VAE (VQ-VAE)

What: Discrete latent space via a learned codebook (vector quantization). Encoder maps input to nearest codebook vectors; decoder uses indices.

Why: Useful when discrete representations are desirable (e.g., symbolic tokens for audio/images).

Applications:

- High-quality generative models for images and audio (e.g., WaveNet + VQ-VAE pipeline).
- Speech synthesis and compression.

Pros/Cons: Enables discrete latent modeling; training requires careful commitment/quantization loss handling.

12) Conditional Autoencoders (cAE / cVAE)

What: Condition encoder/decoder on extra information (labels, side inputs).

Why: Control generation or learn class-conditional embeddings.

Applications: Conditional image generation, controlled speech synthesis, style transfer, conditional anomaly detection.

13) Marginalized / Linear Denoising AEs (mDA / mSDA)

What: For certain noise models and linear encoders, denoising AE can be marginalized leading to closed-form solutions (faster).

Applications: Domain adaptation (mSDA), fast feature learning on tabular data.

Practical applications mapped to AE types (quick reference)

- **Image denoising / inpainting:** Convolutional AE, Denoising AE, VQ-VAE (for high-quality).
- **Anomaly detection (images, time-series):** Undercomplete AE, DAE, Conv-AE, Seq-AE (for sequences), VAE (likelihood-based).
- **Representation learning / embeddings:** Undercomplete, Sparse AE, Stacked AE, Seq-AE.
- **Compression:** Undercomplete AE, Conv-AE, VQ-VAE.
- **Generative modeling / data augmentation:** VAE, AAE, VQ-VAE, conditional variants.
- **Pretraining / transfer learning:** DAE, deep/stacked AE, Conv-AE.
- **Semi-supervised learning / clustering:** AAE, VAE (with mixture priors), cVAE.
- **Speech / audio:** Seq-AE, Conv-AE on spectrograms, VQ-VAE for discrete speech codes.
- **Time-series forecasting & anomaly detection:** Seq-AE (LSTM/GRU), temporal VAE variants.

Practical tips when choosing an autoencoder

1. **Task determines architecture:** Images → Conv-AE; sequences → RNN/Transformer-based AE.
2. **Do you need generation?** If yes, use probabilistic (VAE) or adversarial (AAE/GAN hybrids) models.
3. **Robustness to noise:** Use Denoising or Contractive AEs.
4. **Want interpretable/disentangled latents?** Try β -VAE or structured priors / VQ-VAE.
5. **Anomaly detection:** Simple AEs work but performance depends on whether anomalies are reconstructable; VAEs add likelihood-based scoring.

6. **Compute/data budget:** Simpler AEs are lightweight; deep generative AEs (VAEs, VQ-VAE) require more compute/data.